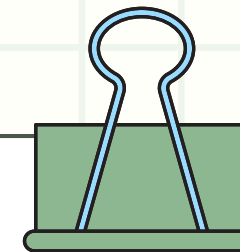




IDE 없이 JAVA 개발해보기



목차

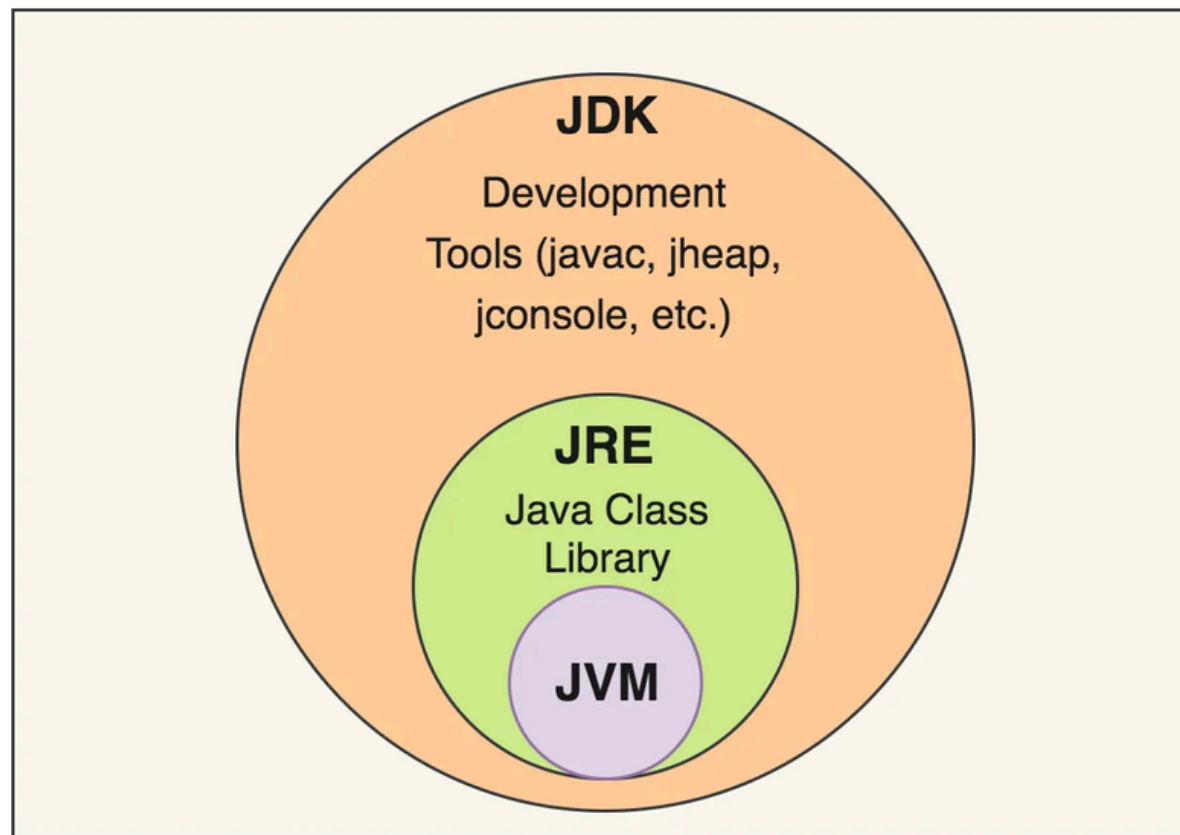
- 01 기본 개념 정리
- 02 실습 1: Hello World
- 03 실습 2: src, bin 분리
- 04 실습 3: 외부 라이브러리
- 05 실습 4: 과제 코드 실행해보기
- 06 정리

01 기본 개념 정리



1. JVM, JDK, JRE
2. 명령어: javac, java
3. Classpath
4. JAR 파일

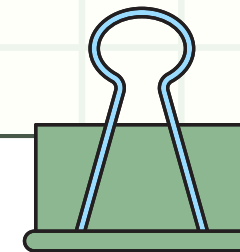
JVM vs. JRE vs. JDK



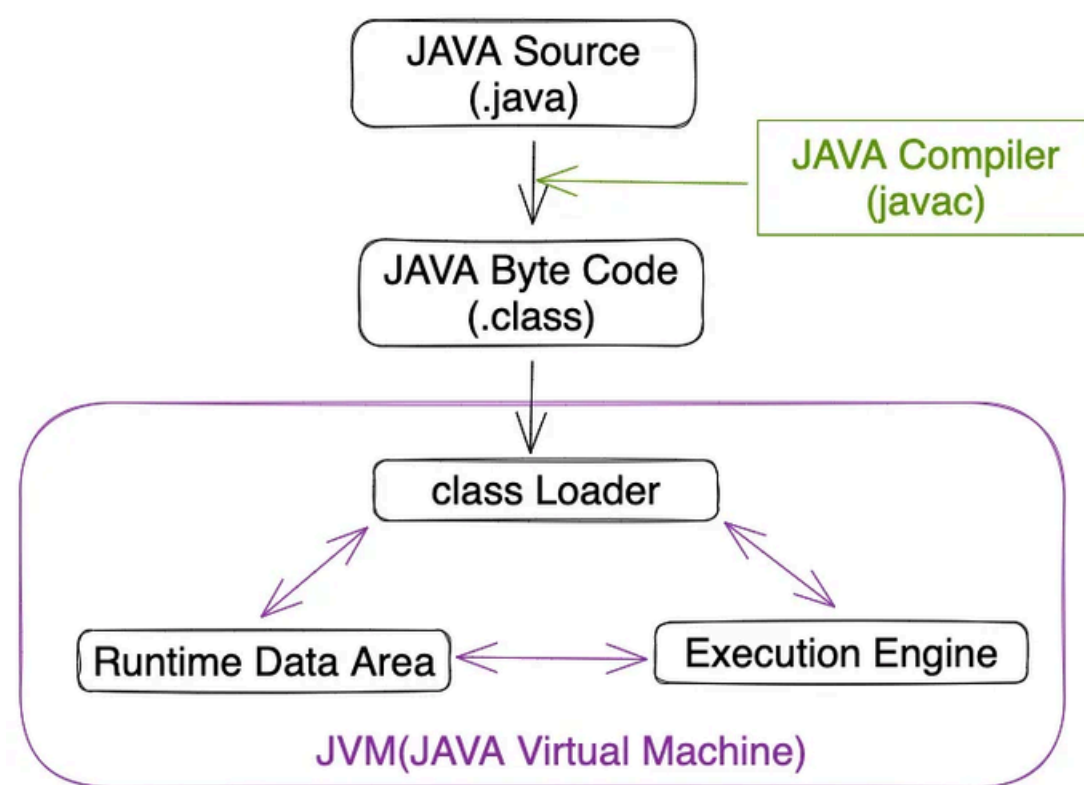
JVM: 바이트 코드를 실행하는 가상 머신

JRE: **자바 표준 라이브러리** 등 실행에 필요한 환경을 갖춘 소프트웨어

JDK: **컴파일러, 디버거** 등의 소프트웨어 개발 도구를 포함하는 JRE 상위 소프트웨어 계층



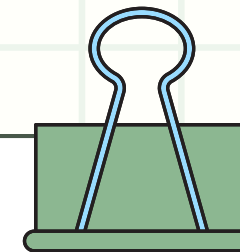
컴파일/실행 명령어



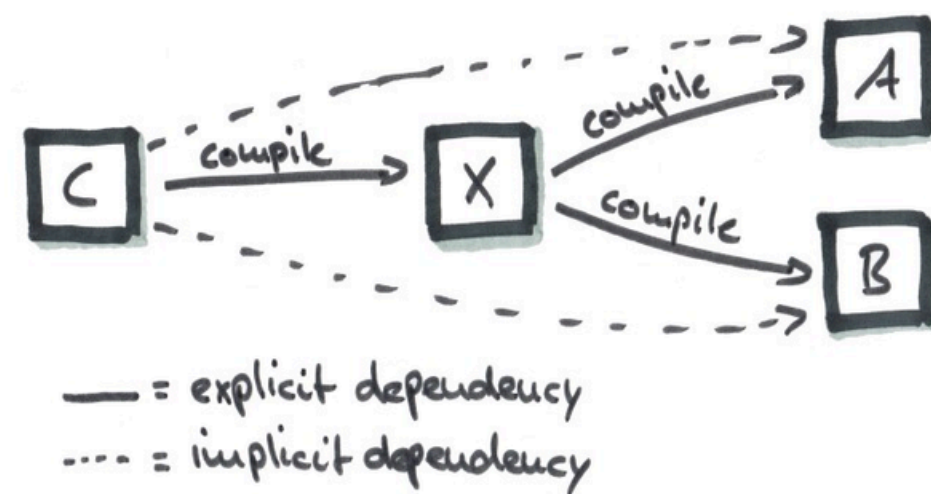
- javac
 - java 파일을 바이트 코드로 컴파일
 - JDK에 포함
- java
 - class 파일을 로드하여 실행
 - JRE에 포함

>_

```
javac MyProgram.java
java MyProgram
```



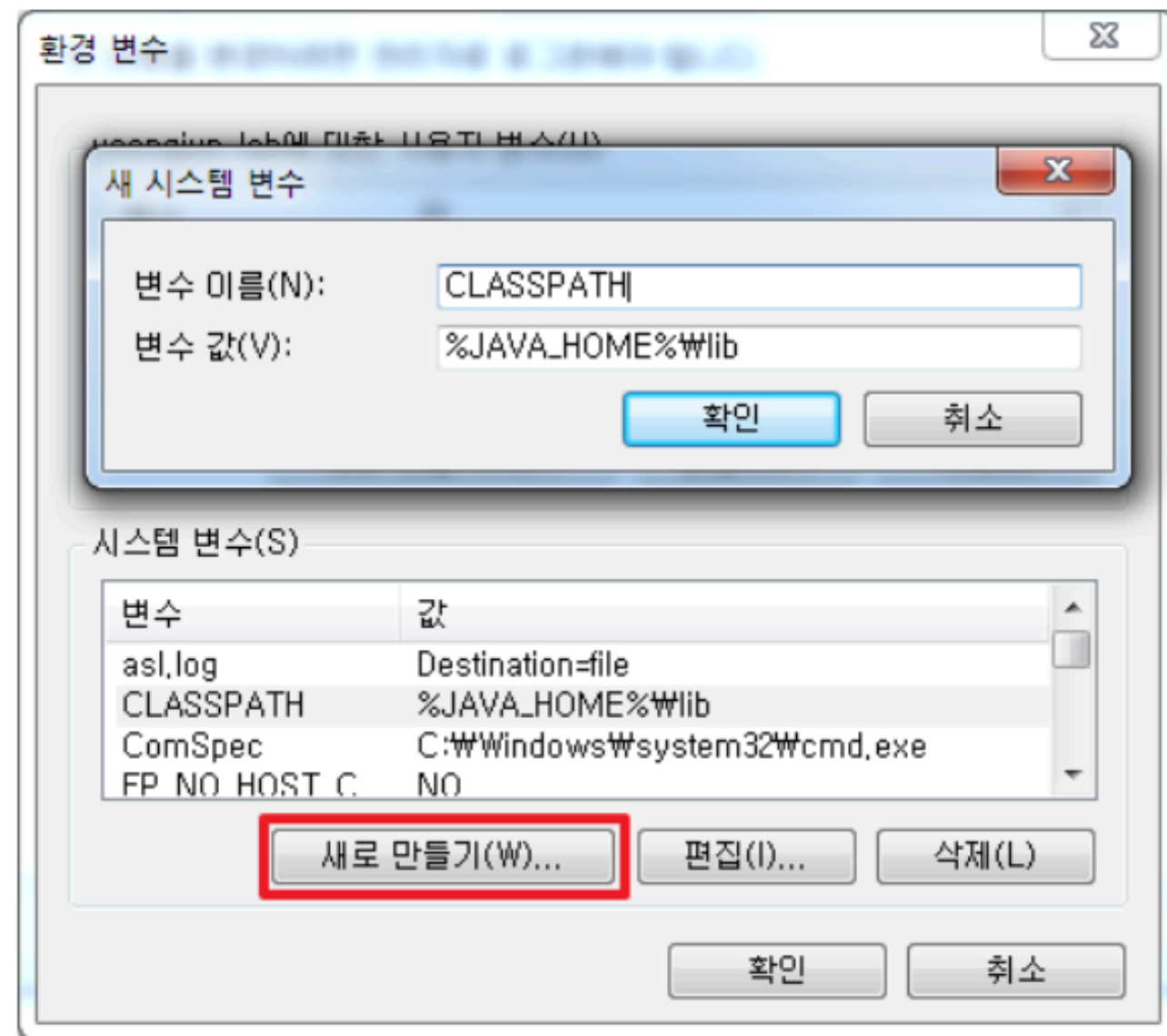
참고: Dependency Compilation



javac로 컴파일할 때 참조되는 다른 클래스도 같이 컴파일된다.



Classpath

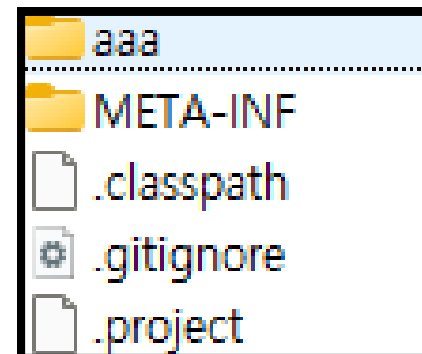
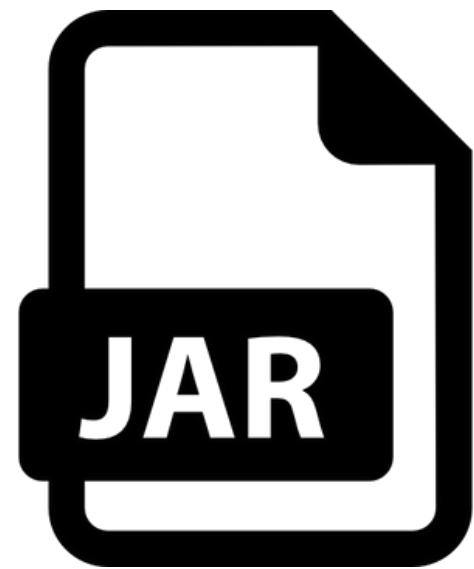


- Java 컴파일 및 실행 시 클래스와 라이브러리를 찾을 때 참고하는 경로 목록
- 특징
 - classpath에 있는 .class만 인식 가능!
 - 기본값: 현재 디렉토리
 - 여러 경로 지정 가능
- 환경변수 등록(Power shell)
 - `$env:CLASSPATH="./lib;./src"`
- 명령줄 삽입
 - `java -cp ./src Main`

Jar 파일



- 바이트코드 파일, 리소스 파일, 메타 데이터를 하나로 묶은 압축 파일



구분	실행 가능 JAR	라이브러리용 JAR
용도	독립 실행 앱 배포	코드 재사용(라이브러리)
Manifest	Main-Class 필수	X
사용 방법	java -jar로 바로 실행	classpath에 .jar 추가
예시	스프링부트 앱, 데스크탑 앱	Jackson, Log4j, JUnit

02 실습 1: Hello World

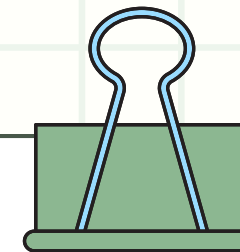


✓ Step 1: 소스코드 작성

```
PS C:\dev\practice\java> notepad Main.java
```

```
class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

실습 링크: <https://github.com/jonghwankoh/java-practice>



✓ Step 2: 컴파일

```
PS C:\dev\practice\java> javac Main.java
PS C:\dev\practice\java> ls
```

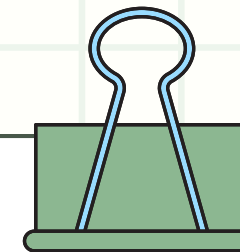
디렉터리: C:\dev\practice\java

Mode	LastWriteTime	Length	Name
-a----	2025-09-10 오전 12:07	413	Main.class
-a----	2025-09-10 오전 12:05	100	Main.java

✓ Step 3: 실행?

```
PS C:\dev\practice\java> java Main
Error: Could not find or load main class Main
Caused by: java.lang.ClassNotFoundException: Main
```

원인: Classpath가 이미 다른 값으로 설정된 경우 발생 가능

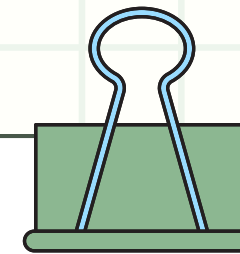


✓ Step 3: 실행 성공

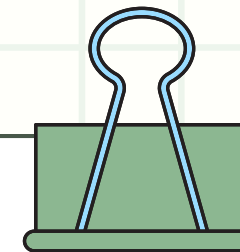
```
PS C:\dev\practice\java> java -cp . Main  
Hello World
```

명령줄에 -cp 옵션 삽입으로 해결

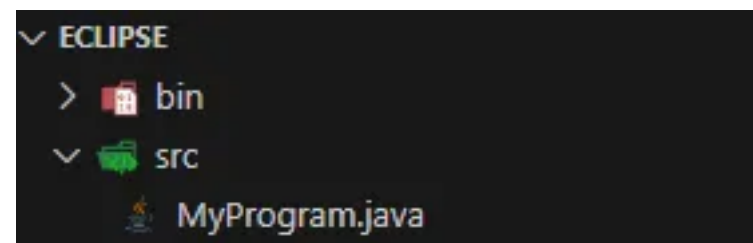
03 실습 2: src, bin 분리 구조



🔄 > 🖥️ > 내 PC > OSDisk (C:) > SSAFY > workspace > samsung01 >				
✂️ 📄 📁 🗑️ 🔍 📄 📄 🗑️ ⬆️ 정렬 ▾ ≡ 보기 ▾ ⋮				
이름	수정한 날짜	유형	크기	
📁 .settings	2025-08-01 오후 5:55	파일 폴더		
📁 bin	2025-08-19 오전 9:22	파일 폴더		
📁 src	2025-08-19 오전 9:22	파일 폴더		
📄 .classpath	2025-08-01 오후 5:55	CLASSPATH 파일	1KB	
📄 .project	2025-08-11 오후 2:11	PROJECT 파일	1KB	



✓ Step 1: 코드 작성



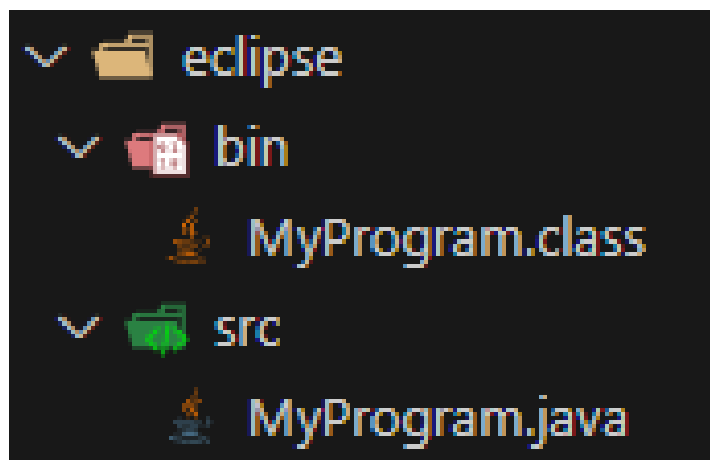
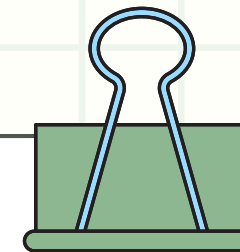
✓ Step 2: -d 옵션으로 컴파일

```
PS C:\SSAFY\workspace\java-practice\eclipse> javac -d bin .\src\MyProgram.java
PS C:\SSAFY\workspace\java-practice\eclipse> ls .\bin\
```

디렉터리 : C:\SSAFY\workspace\java-practice\eclipse\bin

Mode	LastWriteTime	Length	Name
-a----	2025-09-10 오전 10:58	430	MyProgram.class

-d 옵션으로 바이트 코드가 저장될 위치를 지정할 수 있다.

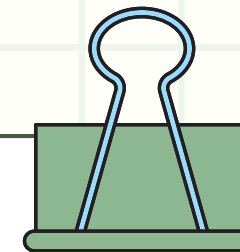


✓ Step 3: -cp 옵션으로 실행

```
PS C:\SSAFY\workspace\java-practice\eclipse> java -cp .\bin MyProgram  
This is my program
```

classpath에 .\bin 등록하여 실행

04 실습 3: 외부 라이브러리 사용해보기



✓ Step 1: 라이브러리 다운로드

Home » com.google.code.gson » gson » 2.8.0

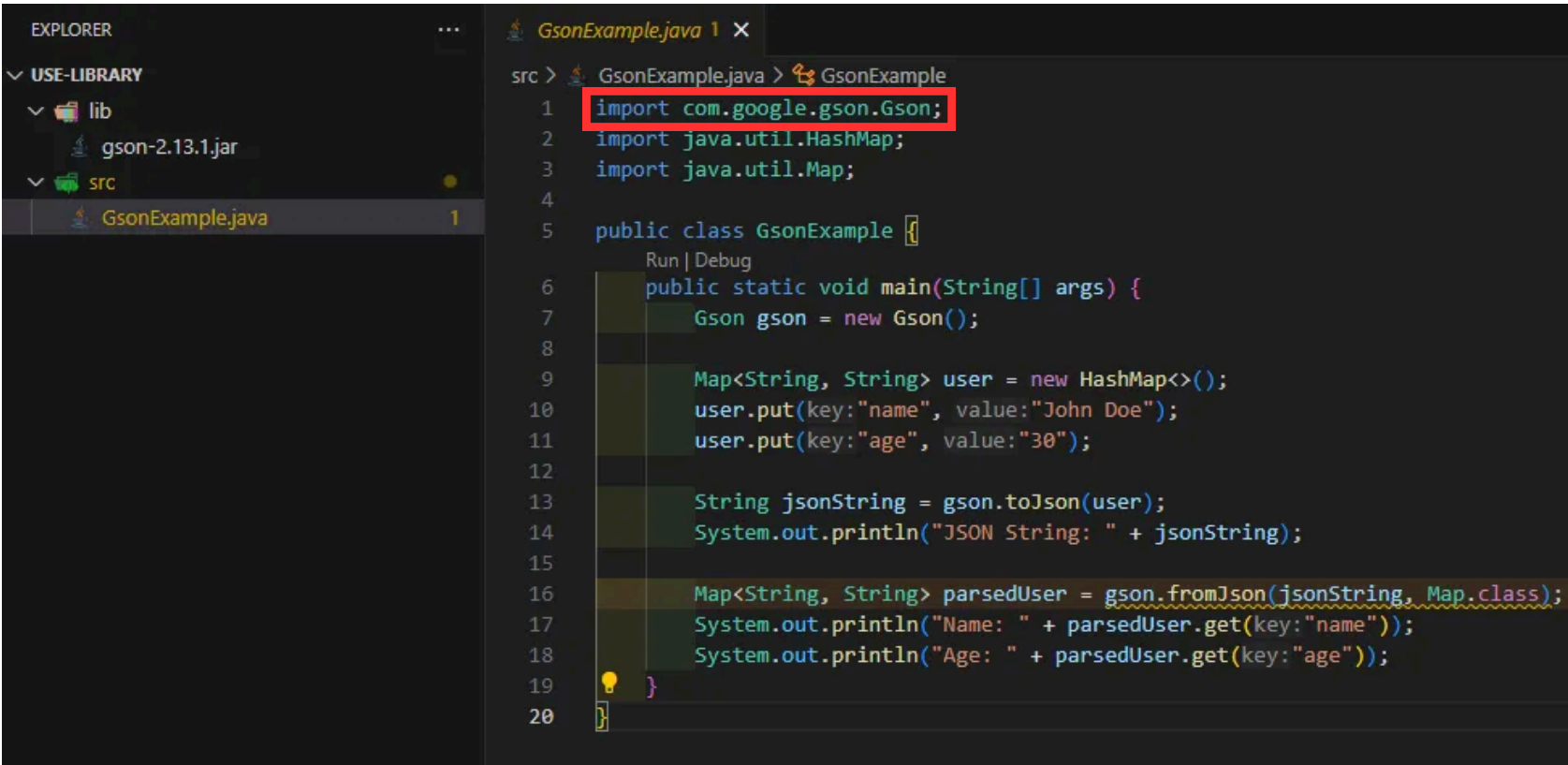
 **Gson » 2.8.0**
Gson

License	Apache 2.0
Categories	JSON Libraries
Date	(Oct 27, 2016)
Files	Download (JAR) (143 KB)
Repositories	Central Sonatype Releases
Used By	3,559 artifacts

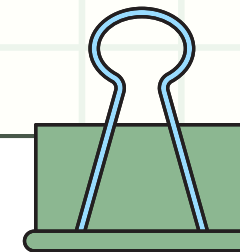
Maven Gradle SBT Ivy Grape Leiningen Buildr

```
<!-- https://mvnrepository.com/artifact/com.google.code.gson/gson -->
<dependency>
  <groupId>com.google.code.gson</groupId>
  <artifactId>gson</artifactId>
  <version>2.8.0</version>
</dependency>
```

✓ Step 2: 코드 작성



```
1 import com.google.gson.Gson;
2 import java.util.HashMap;
3 import java.util.Map;
4
5 public class GsonExample {
6     public static void main(String[] args) {
7         Gson gson = new Gson();
8
9         Map<String, String> user = new HashMap<>();
10        user.put(key:"name", value:"John Doe");
11        user.put(key:"age", value:"30");
12
13        String jsonString = gson.toJson(user);
14        System.out.println("JSON String: " + jsonString);
15
16        Map<String, String> parsedUser = gson.fromJson(jsonString, Map.class);
17        System.out.println("Name: " + parsedUser.get(key:"name"));
18        System.out.println("Age: " + parsedUser.get(key:"age"));
19    }
20 }
```



✓ Step 3: 컴파일

```
PS C:\SSAFY\workspace\java-practice\use-library> javac -cp .\lib\gson-2.13.1.jar .\src\GsonExample.java
Note: .\src\GsonExample.java uses unchecked or unsafe operations.
Note: Recompile with -Xlint:unchecked for details.
```

classpath에 .jar 파일 위치 추가

✓ Step 4: 실행

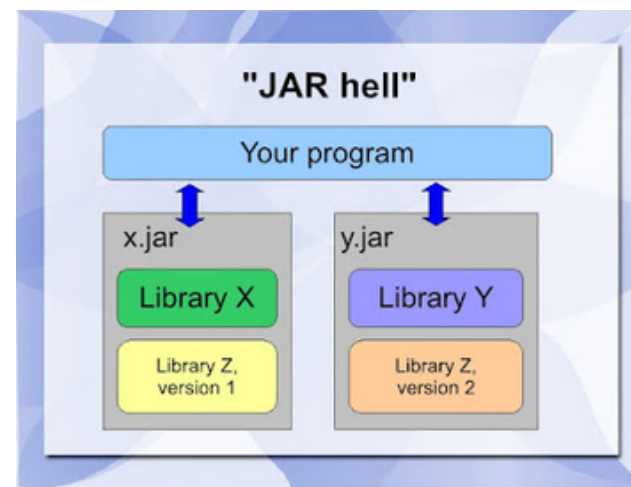
```
PS C:\SSAFY\workspace\java-practice\use-library> java -cp ".\src;.\lib/gson-2.13.1.jar" GsonExample
JSON String: {"name":"John Doe","age":"30"}
Name: John Doe
Age: 30
```

실행시에도 라이브러리 추가

번거로운 라이브러리 관리



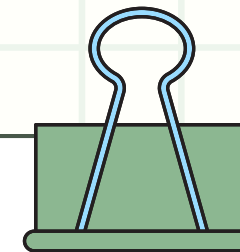
```
PS C:\SSAFY\workspace\java-practice\use-library> java -cp "./src;./lib/gson-2.13.1.jar" GsonExample
JSON String: {"name":"John Doe","age":"30"}
Name: John Doe
Age: 30
```



프로젝트의 규모가 커지고 의존하는 라이브러리가 늘어나면 일일이 classpath 설정을 해줘야한다.

- > 의존성 지옥(JAR Hell)
- > 빌드 자동화 도구가 해결해야 할 핵심 과제

05 실습 4: 과제 코드 실행해보기

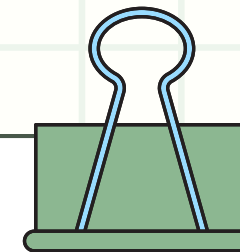


The screenshot shows an IDE with a dark theme. On the left, the 'EXPLORER' panel displays a project structure for 'ALGO_14'. It includes a '.metadata' folder and a series of subfolders numbered 250806 to 250908, followed by 3307 and a folder named '14510'. The '14510' folder is expanded, showing a list of Java files: '.gitkeep', '14510_고종환.java', '14510_권종인.java' (which is selected and has a red error icon), '14510_김성현.java', '14510_김주혜.java', '14510_문승은.java', '14510_박재은.java', '14510_박종현.java', '14510_박창희.java', '14510_서민재.java', '14510_송현우.java', '14510_안효성.java', '14510_윤예슬.java', '14510_정승호.java', '14510_정예준.java', '14510_정청교.java', '14510_조현식.java', and '14510_최종학.java'.

The main editor area shows the code for '14510_권종인.java'. The code is as follows:

```
250909 > 14510 > 14510_권종인.java > ...
4  public class Main { The public type Main must be defined in its own file
5  public static void main(String[] args) throws IOException {
8      int test_case = Integer.parseInt(br.readLine());
9
10     for (int i = 1; i <= test_case; i++) {
11         int N = Integer.parseInt(br.readLine());
12         int[] namu = new int[N];
13
14         StringTokenizer st = new StringTokenizer(br.readLine());
15         for (int j = 0; j < N; j++) {
16             namu[j] = Integer.parseInt(st.nextToken());
17         }
18
19         Arrays.sort(namu);
20
21         int max = namu[N - 1];
22         int sum = 0;
23         int oddcnt = 0;
24         for (int j = 0; j < N; j++) {
25             int d = (max - namu[j]);
26             sum += d;
27             if((d&1) == 1) oddcnt++;
28         }
29
30         int day = 0;
31         while (true) {
32             // day일까지 줄 수 있는 최대 총 성장량
33             int give = (day / 2) * 3 + (day % 2);
34             // day일까지 1을 줄수 있는(oddcnt)의 갯수
35             int ones = (day + 1) / 2;
36
37             if(give >= sum && ones >= oddcnt) break;
38             day++;
39         }
40
41         System.out.println("#" + i + " " + day);
42     }
43 }
44 }
45 }
46 }
```

At the bottom, the 'TERMINAL' panel shows a PowerShell prompt: 'PS C:\SSAFY\git\algo_14> '.



✓ Step 1: 임시 공간에 컴파일

```
javac -d C:\SSAFY -encoding UTF-8 .\250909\14510\14510_권종인.java
```

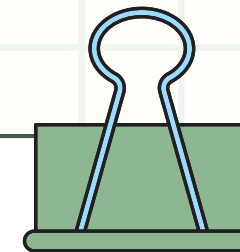
목적지 설정

인코딩 설정

✓ Step 2: 실행

```
java -cp C:\SSAFY\ Main
```

classpath 설정



```
PS C:\SSAFY\git\algo_14> javac -d C:\SSAFY -encoding UTF-8 .\250909\14510\14510_권종인.java
.\250909\14510\14510_권종인.java:4: error: class Main is public, should be declared in a file named Main.java
public class Main {
      ^
1 error
      public 키워드 삭제 후 재시도
PS C:\SSAFY\git\algo_14> javac -d C:\SSAFY -encoding UTF-8 .\250909\14510\14510_권종인.java
PS C:\SSAFY\git\algo_14> java -cp C:\SSAFY Main
1
3
1 2 3
#1 2
```

public class는 **대표 클래스**의 성격을 갖는다.

→ 컴파일러 차원에서 **파일명**과 **public class의 이름**이 같아야한다는 규칙이 존재

→ public class 대신 class로 변경



Tip Single-File Source-Code Programs

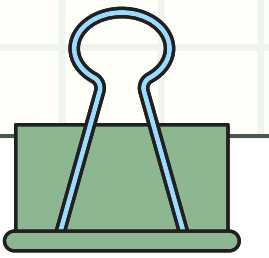
Java 11부터 추가된 기능으로,
마치 python 같은 스크립트 언어처럼 바로 실행하는 것처럼 동작하는 기능

```
PS C:\SSAFY\git\algo_14> java .\250909\14510\14510_권종인.java
1
3
1 2 3
#1 2
PS C:\SSAFY\git\algo_14> █
```

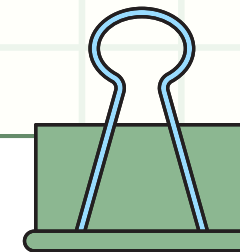
주의: 실제 클래스명이 한글이면 컴파일 에러 발생

- 주요 특징
 - 자동 컴파일: 컴파일-실행 자동화
 - 메모리 실행: 바이트코드 파일을 디스크에 저장하지 않음
 - 메인 클래스 지정: 내부에 있는 단일 public static void main(String[] args)을 갖는 클래스를 엔트리 포인트 설정
- 단점
 - 여러 파일의 컴파일 및 의존성 관리에는 부적합
 - 외부 라이브러리 추가가 제한됨.

06 정리



- 컴파일과 실행 분리
 - javac(컴파일)과 java(실행)의 역할이 다르며, -cp, -d 옵션으로 경로를 제어한다.
- Classpath의 이해
 - JVM은 클래스를 찾을 위치를 classpath로 결정한다.
 - 경로/라이브러리 설정이 올바르지 않으면 ClassNotFoundException 발생한다.
- 외부 라이브러리와 JAR Hell
 - 외부 라이브러리를 개발자가 직접 관리하기는 어렵다.
 - 의존성 관리 도구(Maven/Gradle)가 이 문제를 해결한다.
- 단일 파일 실행 모드(Single-File Source Code Program)
 - java Hello.java 한 줄로 실행 가능하다.
 - 자동 컴파일 + 메모리 실행



감사합니다

THANK
You!